2025 IEEE International Symposium on Workload Characterization (IISWC) | 979-8-3315-4917-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/IISWC66894.2025.00044

XRSight: An End-to-End Hardware–Software Co-Design Platform for XR SoC Evaluation

Prashanth Ganesh
University of California, Berkeley
Berkeley, California, USA
prashanthcganesh108@berkeley.edu

Zekai Lin
University of California, Berkeley
Berkeley, California, USA
zekailin00@berkeley.edu

Yakun Sophia Shao
University of California, Berkeley
Berkeley, California, USA
ysshao@berkeley.edu

***Abstract*—The increasing complexity and performance requirements of extended reality (XR) platforms necessitate a full-stack approach to optimize system architectures. We present XRSight, a hardware–software co-design framework for developing and characterizing extended reality (XR) workloads on embedded systems-on-chip (SoCs). This open-source tool integrates a cycle-accurate, FPGA-accelerated simulation environment with a representative XR application suite to enable architectural exploration and detailed performance analysis. By bridging real-world XR workloads with a modular system-on-chip (SoC) platform, our framework offers fine-grained visibility into SoC execution behavior across compute, memory, and interconnect subsystems. We demonstrate the utility of our work through case studies that examine how different hardware configurations and workload mappings affect end-to-end system performance. Our results show how the tool can inform hardware design decisions and workload optimizations. This framework lowers the barrier to entry for researchers and architects exploring XR-centric system designs in an open and extensible SoC ecosystem.**

***Index Terms*—extended reality, system-on-chip, hardware-software co-design**

I. Introduction

Extended Reality (XR) is poised to reshape the digital landscape, with transformative applications across entertainment, business, personal computing, medicine, and beyond [13]. However, XR imposes a unique combination of power, performance, and area constraints that challenge the capabilities of current personal computing systems. To deliver seamless experiences, XR platforms must achieve motion-to-photon (MTP) latencies under 20 ms within power budgets of 1W or less, all while maintaining a lightweight and comfortable form factor [7], [14]. Moreover, unlike typical personal computing devices that handle only a few real-time tasks, XR systems run multiple computationally intensive kernels (e.g. tracking, estimation, rendering), each critical to user experience. Compounding these challenges, modern XR workloads increasingly integrate deep learning models into core processing pipelines, and the growing ubiquity of large language models (LLMs) is expected to further expand the computational demands of XR applications [17], [29]. Addressing the unique demands of XR requires research and development across multiple layers of the computing stack from hardware components (sensors and advanced display technologies), to SoC design (architectures and high-bandwidth, low-latency memories), to algorithmic innovations (rendering, tracking, machine learning), and more.

While new software developments and custom accelerator designs are essential to keeping pace with the rapidly evolving demands of XR workloads, the heterogeneous and resource-constrained nature of XR systems makes SoC-level design space exploration (DSE) and tradeoff analysis particularly challenging. Prior work has attempted to address these challenges through a variety of approaches, including end-to-end software testbeds [12], [17], domain-specific SoC designs [24], [25], analytical design space exploration tools [4], [23], [28], and dynamic scheduling frameworks [21], [34]. However, XR components operate under tight constraints that cannot be addressed adequately by hardware or software in isolation. Moreover, XR workloads are also highly rendering-intensive, making GPU behavior a critical factor in system performance. However, modeling GPU pipelines in SoC-level simulation remains a challenge due to their complexity and lack of fast and flexible evaluation infrastructure. As a result, a hardware–software co-design tool that enables researchers and architects to jointly iterate on RTL-level SoC design and algorithm development is essential. Such a framework allows for rapid prototyping, what-if analysis of architecture and software changes in tandem, and fine-grained observability into how compute and memory subsystems interact under realistic XR workloads.

To address this need, we present XRSight[1], a new co-design framework for characterizing and analyzing end-to-end XR workloads on embedded SoCs. XRSight extends ILLIXR, a state-of-the-art open-source XR software testbed [12], with a mature SoC design framework to enable joint exploration of the hardware and software design space. To the best of our knowledge, this is the first open-source tool to integrate a comprehensive XR workload with heterogeneous SoCs, enabling evaluation of both architectural and algorithmic design decisions through detailed simulation and characterization. This work makes the following key contributions:

- We develop XRSight, an end-to-end hardware-software co-design platform for XR SoC evaluation. XRSight extends the ILLIXR runtime to execute on heterogeneous SoCs comprising general-purpose CPUs, vector DSPs,

[1]XRSight is open-sourced at https://github.com/ucb-bar/xrsight

2835-2238/25/$31.00 ©2025 IEEE
DOI 10.1109/IISWC66894.2025.00044

GPUs, and custom accelerators, enabling full-system, cycle-accurate RTL simulation on FPGA.
- We develop a flexible GPU modeling framework using a custom FPGA-to-host bridge that enables co-simulation of XR rendering and compute workloads. This infrastructure also supports integration of black-box modules, e.g., external accelerators, alongside the RTL SoC, enabling fast and modular system evaluation.
- We systematically evaluate the effects of different architectural configurations and kernel-to-hardware mappings of end-to-end XR workloads using cycle-accurate simulation, providing detailed insight into how co-design decisions influence both fine-grained kernel behavior and system-level performance.

Together, these contributions represent an important step toward a robust, full-stack evaluation platform for co-designing the next generation of XR-specific SoCs and algorithms.

II. Background and Motivation

With the rise of domain-specific accelerators, domain-specific SoCs have emerged as a promising solution for meeting the performance demands of complex workloads operating under tight power and area constraints. However, current designs, tools, and frameworks fall short in providing architects with full visibility into prototype XR SoC designs or in enabling joint evaluation of hardware IP and algorithmic changes. This section reviews prior work in XR hardware design, design space exploration tools, and testbeds, and highlights the need for a methodology that supports end-to-end XR workloads on a fully configurable, open-source SoC design framework.

A. Testbeds and Workloads for XR

At the software level, several key open-source efforts have emerged in recent years that further highlight the need for a hardware–software co-design framework for XR. XRBench [17] is an open-source machine learning benchmark suite that models real-time multi-task, multi-model (MTMM) inference scenarios. It introduces new metrics to capture XR-specific performance characteristics and system heterogeneity, emphasizing the need for methodologies that jointly optimize SoC architectures and workloads to meet XR's stringent requirements. ILLIXR (Illinois Extended Reality) [12] is an open-source XR runtime and research testbed. It is designed to enable research in real-time XR systems by offering a modular, end-to-end software framework. ILLIXR includes core components of XR systems such as visual-inertial odometry (VIO), IMU integration, timewarp, application rendering, hand tracking, and more. These are connected with an extensible runtime with a ROS-like interface that allows users to add new plugins. In this work, we build on ILLIXR's infrastructure to construct the primary XR workload suite used in evaluating different SoC designs.

B. Accelerators for XR

With the growing prevalence of machine learning in XR platforms, researchers have proposed XR-specific neural network accelerators, including compute-in-memory architectures [26] and 3D-stacked designs for both neural network and image signal processing accelerators [35], [36]. Given the central role of rendering in XR, several hardware designs have been proposed to support real-time execution of techniques such as Neural Radiance Fields [18], [19], Gaussian-based rendering [37], and ray-tracing [10]. These accelerators demonstrate substantial gains in rendering throughput while preserving visual quality. However, these accelerators only evaluate the impact of their design on the specific kernels they address, without the framework to integrate them into a full hardware-software system. The POLO accelerator and algorithm [32] present a co-designed system for gaze segmentation and saccade detection. The evaluation is conducted using GPU simulation, and end-to-end latency is estimated by aggregating the individual latencies of each component. While this represents a meaningful step toward co-design methodologies for XR, it does not offer a framework for evaluating full-system performance in a cycle-accurate SoC simulation environment.

C. Domain-Specific SoC Designs for XR

Several prior efforts have recognized the need for domain-specific SoC designs tailored to XR workloads. ArchiMEDES [24] couples a cluster of RISC-V cores with a configurable DNN accelerator, while Siracusa [25] integrates a RISC-V DSP cluster, an embedded neural engine, and on-chip MRAM in a near-sensor XR SoC. These designs represent specific points in the XR hardware design space but focus primarily on individual deep learning kernels rather than full-system, end-to-end workloads. Moreover, they do not support design space exploration or hardware–software co-design to investigate alternative architectural choices. Other related efforts include system-level SoC designs for AI-driven XR applications [23], [27], scheduling frameworks for multi-modal AI workloads across chiplets [21], and co-design techniques for cooperative rendering [34].

D. Design Space Exploration (DSE) for XR

There is also prior work focused on DSE with applications to XR. Farsi [4] introduces an early-stage system-level simulator evaluated on XR kernels such as CAVA, audio processing, and edge detection. Similarly, ARTEMIS [22] provides an SoC design discovery framework based on workload specifications, also demonstrated with CAVA. Other tools, such as [9], estimate power consumption in distributed on-sensor XR architectures, while [11] supports hardware–software partitioning through virtual prototyping. While these frameworks facilitate exploration of system designs driven by workload performance, they operate at a high architectural abstraction, do not offer RTL-level or cycle-accurate simulation, and are not evaluated on system-level workloads. As a result, they lack the fine-grained observability needed to analyze real-world compute and memory behavior in detail. RoSÉ [20] aims to address full-stack co-simulation

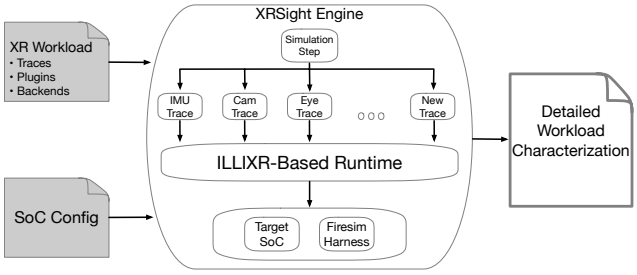


Fig. 1: An overview of the XRSight process. Given an ILLIXR-based XR software workload and SoC architecture, XRSight provides detailed characterization of SoC and system-level metrics for analysis.

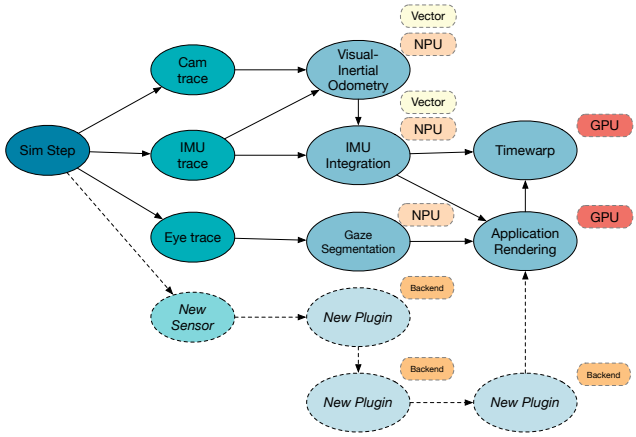


Fig. 2: Key kernels of the baseline XR workloads in ILLIXR and how they can be mapped onto different hardware components. New plugins can be integrated into the base XRSight-ILLIXR workload and compiled to use different SoC functional units.

with pre-silicon hardware, but focuses on robotics applications and evaluates fewer real-time kernels than are present in XR.

Despite these advances, there remains a critical need for a full-stack, open-source framework that combines realistic XR workloads, heterogeneous SoC architectures, and cycle-accurate simulation infrastructure to enable researchers to jointly explore hardware and software design choices with fine-grained visibility and system-level fidelity. To address this gap, we present XRSight.

III. The XRSight Framework

A. Overview

XRSight is a full-stack XR evaluation framework that bridges a custom XR workload and runtime with SoC implementation, evaluated using FPGA-accelerated RTL simulation. We detail the SoC hardware, simulation infrastructure, and workload design we integrate in XRSight (Figure 1) to enable comprehensive co-design, optimization, and cycle-accurate analysis across the entire SoC hardware-software stack.

A key challenge of evaluating ILLIXR in RTL simulation is that ILLIXR, originally designed for real-time execution, cannot be directly deployed on RTL-simulated SoC hardware. Its architecture assumes ample hardware resources, typically only possible on real hardware, to support multiple concurrent, resource-intensive threads with strict timing dependencies. These assumptions break down in RTL simulation environments, due to the slow RTL simulation speed, especially when modeling resource-constrained systems. As a result, when evaluating ILLIXR directly on RTL simulation, critical issues emerge, such as plugins not being scheduled correctly, which leads to incorrect or incomplete execution.

To address this, we modify both the ILLIXR runtime and inter-plugin communication to support execution in discrete timesteps, effectively transforming the system into a *trace-driven simulator* and decoupling ILLIXR from wall-clock time. Since the IMU operates at the highest frequency among system components, we align simulation timesteps with offline IMU sample intervals. This approach enables more representative and tractable analysis, bridging the gap between real-time XR workloads and cycle-accurate SoC simulation.

The primary inputs to XRSight are a description of the XR workload (Section III-B) and the SoC configuration (Section III-C). XRSight integrates these into an engine that drives the simulation as described, and maps it onto the simulation infrastructure (Section III-D). The baseline SoC and runtime are instrumented with RISC-V Hardware Performance Monitor Event registers to collect a range of microarchitectural statistics, including instruction breakdowns, detailed plugin cycle counts, cache and memory performance and control hazard events.

B. Workload Development and Mapping

ILLIXR includes a number of key kernels that we select to construct the baseline XR workload used in XRSight (detailed in Section IV-B). As a trace-driven framework, the user-defined workload must specify offline sensor traces that drive new or existing plugins, similar to how ILLIXR originally provides offline traces for IMU and camera measurements to support VIO and IMU integration. In our evaluation, we extend this by introducing a gaze segmentation plugin that utilizes a new offline eye image trace. Each trace and plugin must define timing dependencies (realized with the decoupled simulation timesteps) to ensure correct scheduling within the simulation loop. Additionally, certain plugins may require custom compilation steps, for example, to support vector instructions or alternate compute backends (see Section IV-B2).

C. SoC Configuration

We elect to use RISC-V as the target architecture for developing XRSight due to its emphasis on efficiency, low power consumption, and open-source ecosystem. These qualities make it particularly well-suited for researchers and system architects to prototype and evaluate designs for emerging workloads such as XR. Moreover, insights gained from RISC-V-based analysis can be generalized to other embedded architectures.

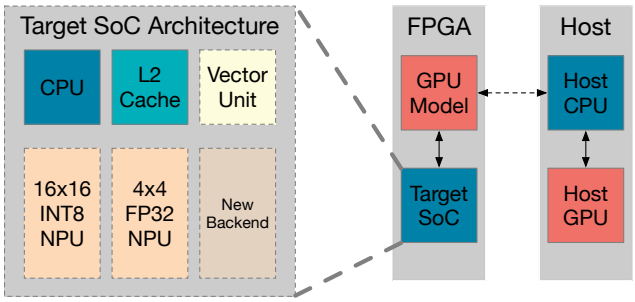


Fig. 3: Architects provide an SoC specification (i.e. Chipyard-built configuration) that is connected to the FPGA-Host simulation infrastructure.

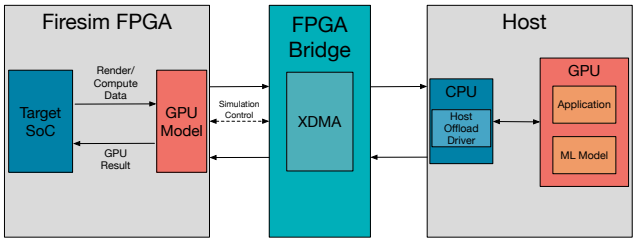


Fig. 4: GPU Model Architecture. We develop a custom FPGA-to-host bridge that enables the simulated SoC to offload rendering tasks to a host GPU, supporting realistic, performance-aware XR pipeline simulation without requiring full RTL GPU integration.

Specifically, we use Chipyard [1], an open-source SoC design framework built around the RISC-V instruction set architecture to enable highly configurable hardware design and generation. Chipyard enables architects to build SoCs with various components like CPUs, caches, accelerators, peripherals, and memory systems, along with support for custom hardware via the TileLink protocol and RoCC interface. Chipyard includes a number of pre-designed parameterizable IP block generators such as Rocket (an in-order CPU core), Saturn vector unit, Gemmini systolic array generator, and more. We leverage several of these blocks to build the different SoC configurations evaluated in this work.

D. Simulation Infrastructure

To support cycle-accurate, full-system evaluation of XR workloads on heterogeneous SoC architectures, XRSight builds on a modular simulation infrastructure, as shown in Figure 3. This infrastructure is designed to bridge the gap between realistic XR execution demands and the limitations of RTL-level simulation, enabling accurate modeling of compute, memory, and offloaded operations. By leveraging FPGA-accelerated simulation and a novel host-bridge interface, XRSight can emulate complex components—such as GPUs or proprietary accelerators—as black-box modules, while maintaining tight integration with the target SoC. In this section, we describe the core simulation components, including our use of FireSim and our method for modeling rendering and compute offload with host-based engines.

1) FireSim Background: We use FireSim [15], an open-source, FPGA-accelerated cycle-exact hardware simulation platform.. It allows researchers and developers to simulate complex RISC-V-based systems at near real-time speeds using FPGA boards, such as the Xilinx U250 used in this work. We leverage FireSim to evaluate different SoC configurations and workload mappings and gain fine-grained insights into system performance. In FireSim terminology, the "target" refers to the RTL design being simulated, and the "host" is the host computer (e.g. x86 workstation and FPGA) that executes the simulation. While most of the SoC components can be simulated using FireSim with reasonable performance, XR rendering workloads—typically executed on GPUs—are too complex and slow to model at the RTL level, motivating the need to co-simulate rendering on the host GPU alongside the rest of the SoC on FPGA.

2) GPU and Black-Box Modeling: Rendering is a critical component of any XR system. ILLIXR was initially designed to utilize a GPU that supports standard graphics APIs such as Vulkan. However, this presents challenges in open-source RISC-V environments. While projects like Vortex GPU [30] present options to address this, they primarily target compute workloads and are difficult to integrate with Chipyard.

More importantly, feedback from both academic researchers and industry XR hardware teams reveals a growing consensus: a fully-featured GPU and full graphics API support are often unnecessary for resource-constrained immersive systems. Supporting APIs like Vulkan introduces considerable overhead and complexity, which may not align with the performance or power constraints of XR workloads. Instead, lightweight, task-specific rendering pipelines are increasingly favored.

To address these limitations and enable realistic rendering within a hardware-software co-design framework, we introduce a novel, performance-driven offloading architecture. Specifically, we develop a custom FPGA-to-host bridge that connects a FireSim-simulated RISC-V SoC to the host machine using memory-mapped registers and a dedicated DMA region in the simulated DRAM address space, as shown in Figure 4. FireSim target-to-host bridges are implemented with a 32-bit AXI4-lite bus. They consist of an FPGA-hosted bridge module that interfaces with the simulated target-RTL, and a CPU-hosted bridge driver that can initiate read and write requests over this bus and interact with host applications (e.g. with socket-based communication). This bridge allows XR applications running in simulation to pass pose data and control signals through MMIO to the host via the bridge module. The host-side driver communicates with the simulator and forwards the pose and control signals to a host-based rendering engine over a socket interface.

The host can also read and write buffers to and from the simulated target using Xilinx XDMA. To enable this, we include a dedicated Xilinx XDMA core in the simulation infrastructure that connects to the FPGA DRAM (which serves as the backing DRAM for the FireSim target [3])

and allows the host driver to interface directly with the target memory subsystem. When the simulated SoC issues a rendering request, our custom FireSim driver pauses the simulation clock, allowing the host to execute the rendering task. Upon completion, results and control signals are returned to the simulated SoC through the bridge and XDMA interface. The driver then resumes the clock and injects simulated latency proportional to the time taken for the operation. To more accurately reflect embedded GPU performance, we throttle the host NVIDIA RTX A5000 GPU using *nvidia-smi*, adjusting its core and memory clocks to mimic lower-performance on-chip accelerators. This real-world latency is captured and replayed within FireSim to model realistic timing behavior. We demonstrate the utility of this proposed method in a case study of dynamic foveated rendering 11.

Crucially, this architecture generalizes beyond rendering. The same host-bridge model enables integration of arbitrary host-based compute backends— ranging from high-performance GPUs to architectural simulators such as GPGPU-Sim [2], [16]. This flexibility facilitates mixed-fidelity simulation- if the functionality of the desired IP can be modeled with a set of control signals and DMA transfers, components that are difficult or impractical to simulate at the RTL-level (e.g., wireless offloading or proprietary IP) can be offloaded to the host while maintaining accurate timing interaction with the simulated SoC.

This capability represents a novel contribution to XR SoC evaluation: it enables modular, scalable, and realistic system-level simulation while avoiding the overhead of full hardware GPU integration. Our method allows architects to quickly explore the design space of lightweight accelerators and co-designed pipelines in a full-stack, end-to-end XR context.

IV. Evaluation Methodology

To demonstrate the capabilities of XRSight and validate its utility as a co-design platform, we evaluate a range of XR workloads across multiple SoC configurations using our FPGA-accelerated simulation infrastructure. This section outlines the baseline hardware setups, selected XR kernels, and the mapping strategies used to explore the performance, resource utilization, and architectural trade-offs inherent to real-time XR systems.

A. SoC Configurations

The XRSight framework supports mapping to various Chipyard-configured SoCs. For the baseline used in many of the experiments in Section V, we surveyed modern commercial XR-oriented SoCs and found that they typically integrate CPUs, GPUs, AI accelerators, ISP/DSP blocks, wireless IPs, and custom XR-specific accelerators for critical algorithms. To model these platforms within Chipyard, our baseline configuration (Figure 3) includes a Rocket core—a five-stage, in-order scalar processor implementing the RV64GC ISA—alongside a 512 KiB, 8-way set-associative L2 cache. Additionally, we use the GPU modeling workflow detailed in Section 3.4 to integrate offloaded rendering within the

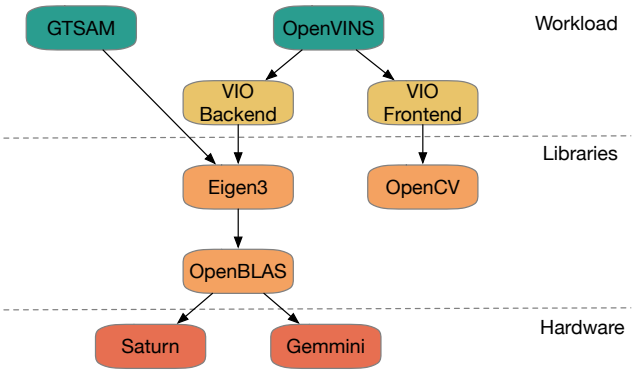


Fig. 5: XRSight VIO Backend Mapping

FireSim simulation. We evaluate the performance impact and tradeoffs of including different configurations of Gemmini, an open-source, full-stack systolic array generator designed for integration into RISC-V-based SoCs built with Chipyard. In addition, we also investigate utilizing the Saturn Vector Unit with vector register length 256 and data path length 128 (supporting the RVV 1.0 extenstion) as a compute backend for VIO (elaborated in Section 3.3.2)

B. Workloads

We select Visual-Inertial Odometry (VIO), IMU integration, pose prediction, asynchronous timewarp, and rendering as the core XR kernels from the set of available ILLIXR plugins, as illustrated in Figure 2. To model emerging ML workloads in XR, we also implement a gaze segmentation plugin using a deep neural network, following a similar approach to ArchiMEDES [24].

1) VIO: VIO is a technique used to estimate the position and orientation (pose) of a moving system—like a robot, drone, or headset—by fusing visual data from cameras with inertial measurements from an IMU (inertial measurement unit, which includes accelerometers and gyroscopes). ILLIXR uses the OpenVINS [8] implementation of VIO. However, while offering more accurate poses, VIO runs at a lower frequency than the display refresh rate (e.g. tuned to 15 Hz in ILLIXR) due to its computational load. Therefore, IMU integration (using the GTSAM algorithm [6]) is used to estimate intermediate poses to obtain a higher frequency at a lower cost. We evaluate tailoring these plugins to map to vector and NPU cores, as detailed in Section IV-B2.

These poses are used downstream in the application rendering engine and asynchronous timewarp- a technique that takes a previously rendered frame and re-projects it based on the user's latest head pose, just before displaying the frame. This makes the scene appear more responsive. [31]. We design these plugins to map to the GPU model backend.

2) BLAS Backend: Accelerating state estimation on our proposed SoC backends requires several careful consideration of the involved abstraction levels and underlying library implementations. OpenVINS and GTSAM both rely on Eigen3 for

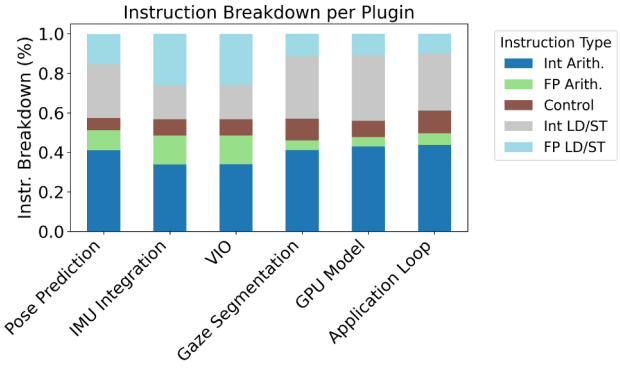


Fig. 6: Breakdown of instruction types across XR plugins, highlighting the heterogeneous compute characteristics of the workload—ranging from control-heavy VIO kernels to compute-intensive rendering and segmentation tasks—informing hardware mapping decisions.

all backend computations, which in turn supports integration with the open-source OpenBLAS [33] linear algebra library. To enable analysis into the impact of co-locating VIO and IMU integration on compute backends, as well as extensibility to map new future plugins in our framework, we elected to interface with the hardware backends at the OpenBLAS level, as shown in Figure 5. This allows us to use the RVV implementation of the core GEMM and GEMV kernels for use with Saturn, or substitute them with calls to an FP32 parameterized Gemmini systolic array.

3) Gaze Segmentation: Modern XR workloads are increasingly reliant on DNN inference to enable complex applications. As detailed in XRBench [17], RITnet [5] is a reference high performance implementation of gaze segmentation. RITnet has 5 down-blocks (each comprised of 5 convolutional layers, LeakyReLU activation, and 2x2 average pooling) and 4 up-blocks (each comprised of 4 convolutional layers, LeakyReLU activation, and 2x2 nearest-neighbor upsampling). We implement RITnet using a Gemmini INT8 systolic array configuration. The systolic array is configured as a 16x16 PE array with 4 scratchpad banks, 2 accumulator banks, and a TLB of size 4.

To tailor the model for Gemmini, we quantize RITnet to INT8, fuse batchnorm layers, use max-pooling instead of average-pooling, and replace LeakyReLU with ReLU for activations. These changes resulted in a negligible drop in inference accuracy but mapped significantly better to Gemmini's architecture. The convolutional layers maintain connections with previous layers through concatenation, which we implement by writing the output of each layer into specific regions of DRAM that could then be read by the next layer. We implement 2x2 nearest neighbor upsampling using deconvolution (transposed convolution).

V. Performance Profiling

We evaluate several SoC and algorithm mapping configurations in XRSight to demonstrate the framework's capabilities and to characterize the plugins described in Section III-B. Prior work has typically examined XR components either in isolation on custom SoC designs or as complete XR workloads on off-the-shelf hardware. In contrast, XRSight enables the evaluation of realistic XR workloads across fully configurable hardware platforms, allowing us to explore different architectural design points.

A. Kernel Characterization

For this study, we evaluate the example ILLIXR mapping shown in Figure 2 on an SoC that integrates an in-order Rocket CPU, INT8 and FP32 Gemmini systolic arrays, and a GPU model as described in Section III-D2. In this configuration, VIO utilizes the FP32 systolic array, Gaze Segmentation utilizes the INT8 systolic array, the application loop (including timewarp and rendering) runs on the GPU model, and the remaining plugins execute on the CPU.

Figure 6 shows the instruction mix per plugin, highlighting the diverse instruction profiles and the balance between compute and memory operations across the workload. Figure 7 reveals key performance trends across the pose prediction, IMU integration, VIO, and gaze segmentation plugins. The data highlights distinct differences in pipeline efficiency and sensitivity to control-flow disruptions across the workload. Among these, VIO emerges as the most performance-critical component, with execution behavior closely tied to branch mispredictions and pipeline flushes, consistent with prior observations in [12]. VIO also exhibits the highest variability in runtime behavior, underscoring the opportunity for further optimization, as explored in Section V-C.

Additionally, we examine the impact of co-locating IMU integration (GTSAM) on the FP32 systolic array alongside VIO. While VIO achieves a notable speedup over both scalar and vector backends when utilizing the systolic array, IMU Integration experiences a significant slowdown compared to its scalar backend (Figure 8). This outcome aligns with the instruction mix shown in Figure 6, where IMU Integration exhibits a relatively low proportion of floating-point arithmetic operations. As a result, offloading to the accelerator introduces more overhead than execution time savings. Notably, Figure 9 shows little performance degradation for VIO when sharing the FP32 systolic array, indicating that the array is not fully utilized by VIO alone. This suggests potential for further backend sharing with other plugins that could benefit from matrix multiplication acceleration.

B. Black-Box/GPU Model Case Study

Foveated rendering is a widely used optimization technique that reduces rendering workload by leveraging the non-uniform acuity of human vision. The fovea, a small region at the center of the retina, provides sharp central vision, while peripheral vision is significantly less sensitive to detail.

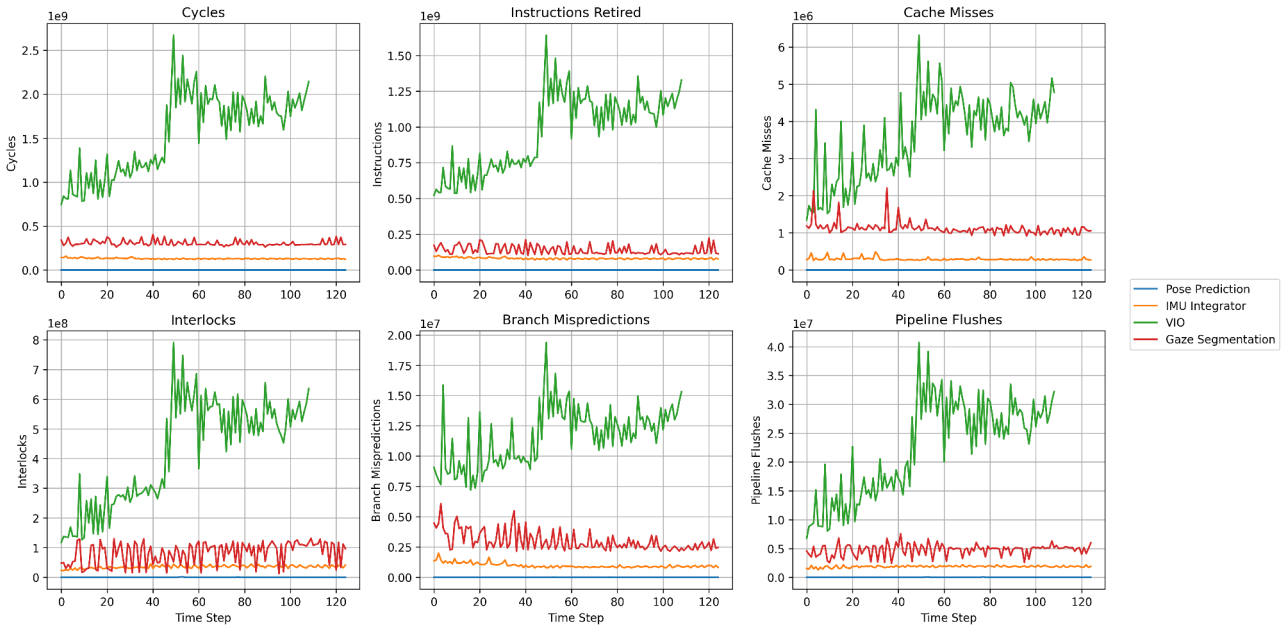


Fig. 7: Performance counter data collected from XRSight reveals microarchitectural behavior across XR plugins, including instruction mix, cache activity, and control hazards, providing insights into performance bottlenecks and optimization opportunities.

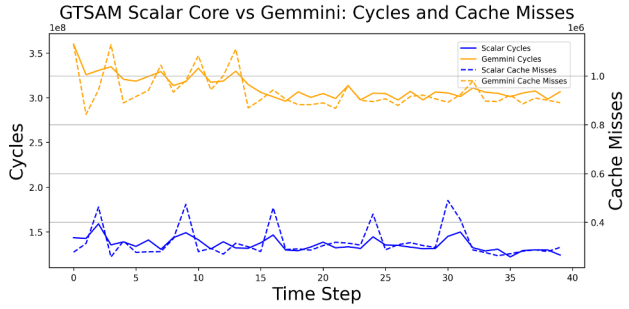


Fig. 8: Comparison of IMU integration performance across scalar and FP32 Gemmini backends, showing that offloading to the systolic array yields actually slowdown due to the low floating-point intensity of the workload.

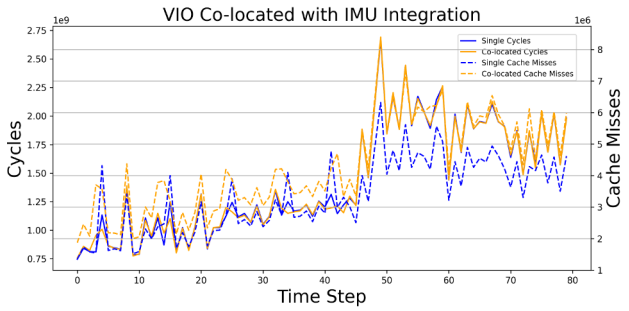


Fig. 9: Co-locating VIO and IMU integration on the FP32 Gemmini backend results in minimal performance degradation for VIO, indicating underutilization of the accelerator and potential for backend sharing across compatible workloads.

Rendering systems can exploit this by generating high-resolution output only within the viewer's gaze region, while reducing fidelity in the periphery—dramatically lowering computational cost without perceptible quality loss. Effective foveated rendering requires both a low-latency gaze tracking mechanism and a graphics pipeline capable of handling mixed-resolution rendering. The XRSight framework enables exploration of this design space by allowing architects to integrate gaze segmentation models on machine learning accelerators (as detailed in Section IV-B3) and feed the dynamically tracked foveal region into a host-driven rendering engine. This provides a flexible and hardware-aware platform for evaluating co-designed gaze-aware rendering pipelines.

To demonstrate the flexibility and utility of our mixed-mode simulation approach for GPU rendering and general black-box compute modeling, we developed a method where the target application running on the simulated SoC dynamically adjusts the fragment shading rate in response to real-time feedback from the GPU model. During the simulation, we use *nvidia-smi* to randomly vary the host NVIDIA RTX A5000 GPU clock between 330 MHz and 420 MHz, to emulate embedded rendering slowdown. Initially, the GPU clock is set to 420 MHz, and the target application collects a baseline average

Fig. 10: Visualization of foveated rendering. The region centered at the fovea is rendered at full resolution, while the surrounding frame is rendered at a lower resolution.

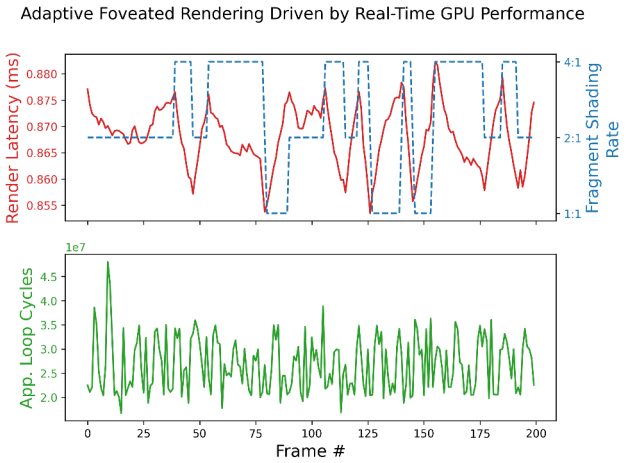


Fig. 11: Demonstration of black-box model flexibility by implementing dynamic foveated rendering based on real-time GPU performance.

rendering time over multiple frames. The rendering plugin then continuously monitors a 10-frame moving average of the render delay. If this average deviates by more than 3% from the baseline, the simulated SoC generates a control signal to adjust the fragment shading rate around the foveal region (obtained through gaze segmentation inference on Gemmini) dynamically.

The Vulkan renderer running on the host supports three shading rate levels: 1:1 (no change), 2:1, and 4:1 (most aggressive). Upon receiving a shading rate adjustment signal, the renderer uses the gaze position computed by the NPU to maintain a small foveal region at full resolution, while reducing sampling in the peripheral regions according to the current shading rate. As shown in Figure 11, this approach maintains rendering latency within a tight bound. The target application responds to spikes in latency—induced by the artificially simulated GPU slowdown—by increasing the shading rate, and returns it to baseline when performance recovers. Additionally, this dynamic adjustment stabilizes the cycles consumed by the application loop, reducing variance due to GPU performance fluctuations.

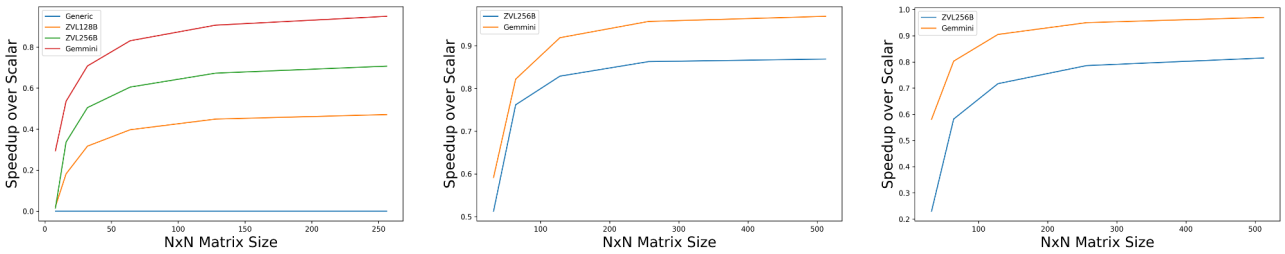
This case study highlights XRSight's capability to enable researchers to prototype and evaluate high-level control strategies using cycle-accurate simulation combined with detailed performance analysis. The example of dynamic foveated rendering based on real-time GPU performance feedback spans multiple interconnected SoC hardware and algorithmic components, illustrating the flexibility of XRSight's black-box modeling approach. It seamlessly integrates heterogeneous models—such as rendering pipeline performance—with cycle-accurate simulation to analyze their end-to-end impact.

C. VIO Case Study

In addition to the broader system-level profiling, we perform a detailed evaluation of OpenVINS on a Chipyard Shuttle core with Saturn vector units and Gemmini accelerators. The VIO pipeline consists of an OpenCV-based frontend for feature detection and a backend that performs pose estimation using the MSCKF algorithm. Linear algebra operations in the backend rely on OpenBLAS, where GEMM and GEMV dominate the computational workload. To optimize these operations, we integrated vectorized RISC-V kernels targeting `vlen=128` and `vlen=256`, as well as a Gemmini-accelerated backend for both GEMM and GEMV. Frame-by-frame cycle counts are collected, starting from the point when the backend is triggered by a sufficient number of tracked features. Performance improvements are measured by comparison with a scalar baseline running the unmodified OpenVINS pipeline.

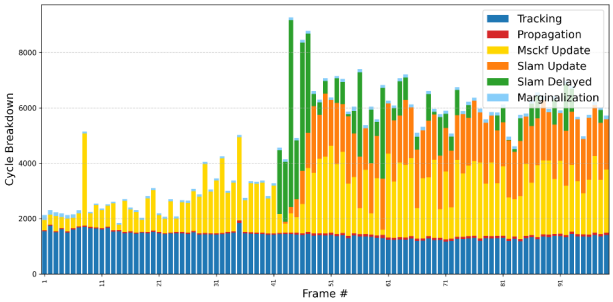
We first profile OpenBLAS kernels in isolation to understand their contribution when integrated into OpenVINS and ILLIXR. As shown in Figure 12a, the GEMM kernel demonstrates increasing speedup with larger matrix sizes. The `vlen=128` vector kernel achieves a 50% speedup (2x fewer cycles) over the scalar baseline, while `vlen=256` reaches 70% (3.3x). The Gemmini backend provides the highest benefit, with a 95% speedup corresponding to a 20x reduction in cycles. For GEMV_T and GEMV_N operations, Figures 12b and 12c shows up to 97% speedup (33.3x) using Gemmini and approximately 85% (6.6x) with the `vlen=256` vector backend.

We also collect per-frame cycle counts across OpenVINS components—tracking, propagation, MSCKF update, marginalization, and SLAM updates. Tracking workloads remain relatively stable across frames, while MSCKF update and propagation stages exhibit variability due to fluctuations in the number of tracked feature points output by the VIO frontend. Marginalization contributes minimal overhead. With the `vlen=256` backend, the average speedup across 200 frames is 8.49%, with peak improvements of up to ~25% in a 9-frame moving average (Figure 14). The performance variation is primarily due to floating-point rounding in Level 3 BLAS routines such as TRSM, SYRK, and SYMM, which depend on GEMM and influence SLAM feature promotion.

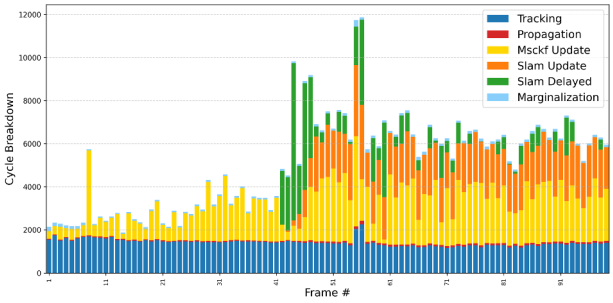


(a) GEMM kernel speedup. (b) GEMV_N kernel speedup. (c) GEMV_T kernel speedup.

Fig. 12: Speedup of GEMM, GEMV_N, and GEMV_T kernels relative to matrix dimensions.



(a) Gemmini backend



(b) Saturn backend `vlen=256`

Fig. 13: VIO component cycle breakdown

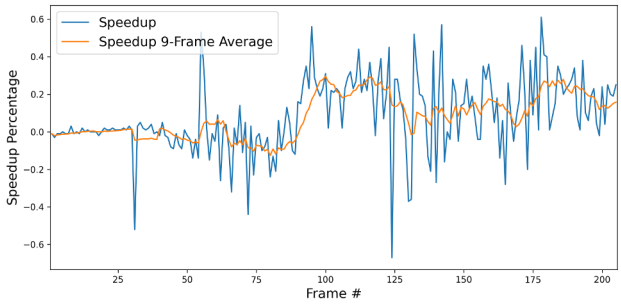


Fig. 14: Speedup of Saturn compared to scalar backend across frames

The Gemmini backend shows more consistent behavior. Although it only accelerates GEMM and GEMV but not other BLAS level 3 operations, it achieves an average 7.7% speedup with less frame-to-frame variation, due to the minimal numerical divergence across non-GEMM BLAS level 3 operations.

Despite high kernel-level gains, the end-to-end speedup is limited by software bottlenecks and numerical variability in the VIO stack. These results underscore the need for hardware–software co-design to realize system-level benefits in real-time VIO workloads.

VI. Conclusion

XR workloads impose stringent power, performance, and area requirements on SoC design that cannot be isolated through hardware or software optimizations alone. XRSight bridges this gap by providing the first open-source, full-stack hardware-software co-design framework for XR-specific embedded SoCs. We demonstrate how to map end-to-end XR workloads onto heterogeneous SoC configurations with vector, systolic-array, and GPU-modeled backends, and evaluate their behavior through FPGA-accelerated, cycle-accurate simulation. In doing so, we illustrate the utility of XRSight in enabling deep insights into the interactions between architectural design choices and algorithmic performance. We also demonstrate the potential of this infrastructure to support both detailed kernel-level analysis and system-level trade-off exploration, offering a critical tool for advancing next-generation XR-specific SoCs and workloads. We are actively expanding this framework to more optimally map multithreaded applications to heterogeneous backends, optimize core XR kernels through a co-design lens, explore emerging workloads such as edge LLM inference, and more. Moving forward, XRSight provides a foundation for researchers to prototype new accelerator architectures, scheduling policies, and optimized XR pipelines in an open and reproducible manner.

Acknowledgment

We would like to thank the anonymous reviewers and artifact evaluators for their insightful feedback. We sincerely thank Kostadin Ilov for his management of the compute infrastructure we used for development.

This research was supported in part by the NSF Awards CCF- 2238346 and in part by SLICE Lab industrial sponsors and affiliates. This project is funded by the Microelectronics Commons Program, a DoD initiative. Distribution Statement A: Approved for public release. Distribution is unlimited. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the U.S. Government.

References

[1] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton *et al.*, "Chipyard: Integrated design, simulation, and implementation framework for custom socs," *Ieee Micro*, vol. 40, no. 4, pp. 10–21, 2020.

[2] A. Bakhoda, G. L. Yuan, W. W. Fung, H. Wong, and T. M. Aamodt, "Analyzing cuda workloads using a detailed gpu simulator," in *2009 IEEE international symposium on performance analysis of systems and software*. IEEE, 2009, pp. 163–174.

[3] D. Biancolin, S. Karandikar, D. Kim, J. Koenig, A. Waterman, J. Bachrach, and K. Asanovic, "Fased: Fpga-accelerated simulation and evaluation of dram," in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2019, pp. 330–339.

[4] B. Boroujerdian, Y. Jing, D. Tripathy, A. Kumar, L. Subramanian, L. Yen, V. Lee, V. Venkatesan, A. Jindal, R. Shearer *et al.*, "Farsi: An early-stage design space exploration framework to tame the domain-specific system-on-chip complexity," *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 2, pp. 1–35, 2023.

[5] A. K. Chaudhary, R. Kothari, M. Acharya, S. Dangi, N. Nair, R. Bailey, C. Kanan, G. Diaz, and J. B. Pelz, "Ritnet: Real-time semantic segmentation of the eye for gaze tracking," in *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*. IEEE, 2019, pp. 3698–3702.

[6] F. Dellaert, "Factor graphs and gtsam: A hands-on introduction," *Georgia Institute of Technology, Tech. Rep*, vol. 2, no. 4, 2012.

[7] M. S. Elbamby, C. Perfecto, M. Bennis, and K. Doppler, "Toward low-latency and ultra-reliable virtual reality," *IEEE network*, vol. 32, no. 2, pp. 78–84, 2018.

[8] P. Geneva, K. Eckenhoff, W. Lee, Y. Yang, and G. Huang, "Openvins: A research platform for visual-inertial estimation," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 4666–4672.

[9] J. Gomez, S. Patel, S. S. Sarwar, Z. Li, R. Capoccia, Z. Wang, R. Pinkham, A. Berkovich, T.-H. Tsai, B. De Salvo *et al.*, "Distributed on-sensor compute system for ar/vr devices: A semi-analytical simulation framework for power estimation," *arXiv preprint arXiv:2203.07474*, 2022.

[10] S. Guo, S. S. Sapatnekar, and J. Gu, "Software-hardware codesign of ray-tracing accelerator for edge ar/vr with viewpoint-focused 3d construction and efficient data structure," in *2024 IEEE 67th International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE, 2024, pp. 267–271.

[11] C. Hazott and D. Grosse, "Dsa monitoring framework for hw/sw partitioning of application kernels leveraging vps," in *DVCon Europe 2023; Design and Verification Conference and Exhibition Europe*. VDE, 2023, pp. 34–41.

[12] M. Huzaifa, R. Desai, S. Grayson, X. Jiang, Y. Jing, J. Lee, F. Lu, Y. Pang, J. Ravichandran, F. Sinclair *et al.*, "Illixr: Enabling end-to-end extended reality research," in *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2021, pp. 24–38.

[13] A. Ivanova, "Vr & ar technologies: opportunities and application obstacles," *Strategic decisions and risk management*, no. 3, pp. 88–107, 2018.

[14] D. Kanter, "Graphics processing requirements for enabling immersive vr, amd dev. whitepaper, 2015, p 1–12," 2018.

[15] S. Karandikar, H. Mao, D. Kim, D. Biancolin, A. Amid, D. Lee, N. Pemberton, E. Amaro, C. Schmidt, A. Chopra *et al.*, "Firesim: Fpga-accelerated cycle-exact scale-out system simulation in the public cloud," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 29–42.

[16] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated gpu modeling," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 473–486.

[17] H. Kwon, K. Nair, J. Seo, J. Yik, D. Mohapatra, D. Zhan, J. Song, P. Capak, P. Zhang, P. Vajda *et al.*, "Xrbench: An extended reality (xr) machine learning benchmark suite for the metaverse," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 1–20, 2023.

[18] C. Li, S. Li, Y. Zhao, W. Zhu, and Y. Lin, "Rt-nerf: Real-time on-device neural radiance fields towards immersive ar/vr rendering," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.

[19] S. Li, C. Li, W. Zhu, B. Yu, Y. Zhao, C. Wan, H. You, H. Shi, and Y. Lin, "Instant-3d: Instant neural radiance field training towards on-device ar/vr 3d reconstruction," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.

[20] D. Nikiforov, S. C. Dong, C. L. Zhang, S. Kim, B. Nikolic, and Y. S. Shao, "Rosé: A hardware-software co-simulation infrastructure enabling pre-silicon full-stack robotics soc evaluation," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.

[21] M. Odema, L. Chen, H. Kwon, and M. A. Al Faruque, "Scar: Scheduling multi-model ai workloads on heterogeneous multi-chiplet module accelerators," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 565–579.

[22] S. Pal, A. Amarnath, B. Boroujerdian, A. Vega, A. Buyuktosunoglu, J.-D. Wellman, V. J. Reddi, and P. Bose, "Artemis: Agile discovery of efficient real-time systems-on-chips in the heterogeneous era," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1320–1334.

[23] V. Parmar, S. S. Sarwar, Z. Li, H.-H. S. Lee, B. De Salvo, and M. Suri, "Exploring memory-oriented design optimization of edge ai hardware for extended reality applications," *IEEE Micro*, vol. 43, no. 6, pp. 40–49, 2023.

[24] A. S. Prasad, L. Benini, and F. Conti, "Specialization meets flexibility: A heterogeneous architecture for high-efficiency, high-flexibility ar/vr processing," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.

[25] A. S. Prasad, M. Scherer, F. Conti, D. Rossi, A. Di Mauro, M. Eggimann, J. T. Gómez, Z. Li, S. S. Sarwar, Z. Wang *et al.*, "Siracusa: A 16 nm heterogenous risc-v soc for extended reality with at-mram neural engine," *IEEE Journal of Solid-State Circuits*, 2024.

[26] H. E. Sumbul, J.-s. Seo, D. H. Morris, and E. Beigne, "A fully digital and row-pipelined compute-in-memory neural network accelerator with system-on-chip-level benchmarking for augmented/virtual reality applications," *IEEE Micro*, vol. 44, no. 2, pp. 61–70, 2023.

[27] H. E. Sumbul, T. F. Wu, Y. Li, S. S. Sarwar, W. Koven, E. Murphy-Trotzky, X. Cai, E. Ansari, D. H. Morris, H. Liu *et al.*, "System-level design and integration of a prototype ar/vr hardware featuring a custom low-power dnn accelerator chip in 7nm technology for codec avatars," in *2022 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 2022, pp. 01–08.

[28] X. Sun, X. Peng, S. Q. Zhang, J. Gomez, W.-S. Khwa, S. S. Sarwar, Z. Li, W. Cao, Z. Wang, C. Liu *et al.*, "Estimating power, performance, and area for on-sensor deployment of ar/vr workloads using an analytical framework," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 6, pp. 1–27, 2024.

[29] Y. Tang, J. Situ, A. Y. Cui, M. Wu, and Y. Huang, "Llm integration in extended reality: A comprehensive review of current trends, challenges, and future perspectives," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–24.

[30] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and K. Hyesoon, "Vortex: Extending the risc-v isa for gpgpu and 3d-graphics," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 754–766.

[31] J. M. P. Van Waveren, "The asynchronous time warp for virtual reality on consumer hardware," in *Proceedings of the 22nd ACM Conference on Virtual Reality Software and Technology*, 2016, pp. 37–46.

[32] H. Wang, W. Liu, K. Chen, Q. Sun, and S. Q. Zhang, "Process only where you look: Hardware and algorithm co-optimization for efficient gaze-tracked foveated rendering in virtual reality," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 344–358.

[33] Q. Wang, X. Zhang, Y. Zhang, and Q. Yi, "Augem: automatically generate high performance dense linear algebra kernels on x86 cpus," in

Proceedings of the international conference on high performance computing, networking, storage and analysis, 2013, pp. 1–12.
[34] C. Xie, X. Li, Y. Hu, H. Peng, M. Taylor, and S. L. Song, "Q-vr: system-level design for future mobile collaborative virtual reality," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 587–599.
[35] L. Yang, C. Kao, S. Srikanth, H. E. Sumbul, T. F. Wu, H. Liu, and E. Beigne, "Characterization and design of 3d-stacked memory for image signal processing on ar/vr devices," in *Proceedings of the International Symposium on Memory Systems*, 2024, pp. 38–44.
[36] L. Yang, R. M. Radway, Y.-H. Chen, T. F. Wu, H. Liu, E. Ansari, V. Chandra, S. Mitra, and E. Beigné, "Three-dimensional stacked neural network accelerator architectures for ar/vr applications," *IEEE Micro*, vol. 42, no. 6, pp. 116–124, 2022.
[37] Z. Ye, Y. Fu, J. Zhang, L. Li, Y. Zhang, S. Li, C. Wan, C. Wan, C. Li, S. Prathipati *et al.*, "Gaussian blending unit: An edge gpu plug-in for real-time gaussian-based rendering in ar/vr," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 353–365.

Appendix

A. Abstract

We evaluate the artifacts of this paper via FPGA-accelerated RTL simulation on Firesim of our proposed XR co-design testbed executing the XR workload as described in V-A. We use RISC-V Hardware Performance Monitor counters to evaluate the microarchitectural behaviors, accelerator co-location impact, and GPU/Black-Box case study discussed in the paper.

B. Artifact check-list (meta-information)

- **Compilation:** XR runtime compiled using RISC-V toolchain, RTL simulated on Firesim FGPA-accelerated RTL simulation.
- **Data set:** EuRoC dataset recorded on a Micro Aerial Vehicle for VIO and OpenEDS Semantic Segmentation dataset for gaze segmentation.
- **Hardware:** Ubuntu 22.04 server with NVIDIA A5000 GPU and Xilinx Alveo U250 FPGA.
- **Metrics:** Cycle count, instructions retired, cache performance, interlocks, branch mispredictions, and pipeline flushes from performance counters.
- **Output:** Plots in PNG detailing workload performance and case study results.
- **Experiments:** RTL simulation of RISC-V SoC running XR workload.
- **How much disk space required (approximately)?:** 175 GB
- **How much time is needed to prepare workflow (approximately)?:** 2 hours.
- **How much time is needed to complete experiments (approximately)?:** 30 hours.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** BSD-3. Each ILLIXR component is under one of ElasticFusion License, NCSA, MIT, Simplified BSD, LGPL v3.0, LGPL v2.1, Boost Software License v1.0, GPL v3.0.
- **Data licenses (if publicly available)?: Each dataset is licensed under one of:** ElasticFusion License, Creative Commons 0., Creative Commons Attribution 4.0 International
- **Archived (provide DOI)?:** 10.5281/zenodo.17051658

C. Description

Our implementation requires the Firesim FPGA-accelerated RTL simulation framework with a Xilinx Alveo U250 FPGA, and an NVIDIA discrete GPU (e.g. A5000). The following setup instructions need to be carried out in a Linux compute server (tested on Ubuntu 22.04) that has access to Xilinx Vivado tools. We provide Linux images containing the XR workload for the RTL simulation. All other code and data will be set up from scratch.

1) How to access: Obtain the artifact tarball from https://zenodo.org/records/17051658. The artifact consists of:

- Chipyard framework that contains the SoC configurations evaluated in XRSight.
- Pre-built FPGA bitstreams from the aforementioned SoC configurations and for the Firesim test harness.
- Linux images containing source code (and compiled binaries) to execute on the XRSight infrastructure.
- Setup, execution, and post-processing scripts to run the experiments and generate the figures.

D. Chipyard and Firesim Installation

1) Extracting artifact: In a linux shell, run the following commands (in the home directory):

```
# download artifact tarball
curl -O "https://zenodo.org/records/17051658/files/
    xrsight-artifact-full.tar.gz?download=1"
tar -xzvf xrsight-artifact-full.tar.gz

# install Conda
cd ~
wget "https://github.com/conda-forge/miniforge/
  releases/latest/download/Miniforge3-$(uname)-$(
      uname -m).sh"

# Select yes when prompted to update profile
bash Miniforge3-$(uname)-$(uname -m).sh
source ~/.bashrc

# Build Chipyard (may take up to 30 minutes)
cd ~/xrsight-iiswc-artifact/xrsight-chipyard
./build-setup.sh riscv-tools -s 5
```

2) Firesim setup: Follow the Firesim local FGPA setup instructions provided at https://docs.fires.im/en/latest/Local-FPGA-Initial-Setup.html

```
# after local FPGA setup
cd ~/xrsight-iiswc-artifact/xrsight-chipyard/sims/
    firesim
source sourceme-manager.sh

firesim managerinit --platform xilinx_alveo_u250
firesim enumeratefpgas

cd ~/xrsight-chipyard/software/firemarshal
./init-submodules.sh
./marshal -v build ubuntu-base.json
./marshal -v install ubuntu-base.json
```

E. Install ILLIXR Host worker

Install dependencies:

```
sudo apt-get install libglew-dev libglu1-mesa-dev
  libsqlite3-dev libx11-dev libgl-dev pkg-config
  libopencv-dev libeigen3-dev libc6-dev libspdlog-
      dev
  libboost-all-dev git cmake cmake-data ccache
  libglfw3-dev libglm-dev libjpeg-dev libusb-1.0.0-
      dev
  libuvc-dev libopenxr-dev libopenxr1-monado libpng-
      dev
```

```
  libsdl2-dev libtiff-dev udev libudev-dev
      libwayland-dev
  wayland-protocols libx11-xcb-dev libxcb-glx0-dev
  libxcb-randr0-dev libxkbcommon-dev
```

Install ILLIXR:

```
cd ~/xrsight-iiswc-artifact/ILLIXR
mkdir build
cd build
```

Build using CMake:

```
cmake .. \
  -DCMAKE_INSTALL_PREFIX=<full_path_to_user_home_dir
      >/illixr-deps/ \
  -DYAML_FILE=profiles/gpu_offload.yaml \
  -DCMAKE_BUILD_TYPE=Release


cmake --build . -j32 && cmake --install .
```

Add the following to `~/.bashrc`:

```
export PATH=$PATH:<full path to user home dir>/
    illixr-deps/bin
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:<full path
    to user
  home dir>/illixr-deps/lib
```

Extract Linux Images:

```
cd ~/xrsight-iiswc-artifact
chmod +x download_images.sh
./download_images.sh
```

F. Experiment workflow

There are 3 shell scripts for setting up the experiments and 3 monitoring scripts to run them and save the output. Each experiment may take 2-3 hours to complete. *Note: It is recommended to log out of the Linux shell and log back in to ensure the environment is clean before each setup.*

1) Experiment 1: VIO: Run the following to setup the Firesim simulation and launch it along with the host-side ILLIXR worker:

```
cd ~/xrsight-iiswc-artifact/
chmod +x setup-exp1.sh
./setup-exp1.sh
```

When the following line appears, the setup is complete (it may continue to print output to stdout).

```
[ILLIXR host server] New client connected (..)
```

Open a new terminal window and run:

```
cd ~/xrsight-iiswc-artifact/
chmod +x monitor-exp1.sh
./monitor-exp1.sh
```

Upon completion, the script will terminate the simulation and the host worker. You can verify the creation of a new file named `screen_dumps/ov-gemm.txt`

2) Experiment 2: Eye Tracking: The instructions are identical to those of Experiment 1, using `setup-exp2.sh` and `monitor-exp2.sh`

Upon termination, you can verify the creation of a new file named `screen_dumps/et-scalar.txt`.

3) Experiment 3: GTSAM: The instructions are identical to those of Experiments 1 and 2, using `setup-exp3.sh` and `monitor-exp3.sh`

You can verify the creation of a new file named `screen_dumps/ov-gtsam-gemm.txt`.

4) Experiment 4: VIO Case Study: As in the earlier experiments, use the following scripts to run the VIO case study experiments. Note that the generic and Saturn experiments may take 2-3 hours each, and the Gemmini experiment may take 10-12 hours.

```
./setup-vio-generic.sh
./monitor-vio-generic.sh
```

Upon completion, you can verify the creation of a new file named `screen_dumps/vio_generic.txt`

```
./setup-vio-saturn.sh
./monitor-vio-saturn.sh
```

Upon completion, you can verify the creation of a new file named `screen_dumps/vio_saturn.txt`

```
./setup-vio-gemmini.sh
./monitor-vio-gemmini.sh
```

Upon completion, you can verify the creation of a new file named `screen_dumps/vio_gemmini.txt`.

G. Evaluation and expected results

Once the first 3 experiments are complete, run the following:

```
cd ~/xrsight-iiswc-artifact
python parse.py
```

In the `figures` directory, there will be 4 new plots. Figure 7 corresponds to `counters_over_time.png`. Figure 8 corresponds to `gtsam_comparison.png`. Figure 9 corresponds to `ov_comparison.png`. We expect these to match the reported figures in the paper closely.

Additionally, Figure 11 corresponds to `gpu_case_study.png`. We do not expect this to match Figure 11 in the paper, as it involves randomly varying the host GPU clock speeds using `nvidia-smi`. Without assuming requisite permissions to modify the host GPU clock, the script does not recreate the exact variation performed when reporting the results, but the underlying behavior of the GPU model with the dynamic foveation rate is still evident.

Once the VIO experiments are complete, run the following:

```
cd ~/xrsight-iiswc-artifact
python parse_vio_full.py
```

This will produce plots corresponding to Figure 12 (`*_kernel_speedup.png`), Figure 13 (`gemmini_bars.png` and `saturn_bars.png`), and Figure 14 (`ov_speedup_over_scalar.png`). We also provide `parse_vio_saturn_only.py` to generate plots without the more time-consuming Gemmini VIO experiment.

H. Methodology

Submission, reviewing and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-and-badging-current
- https://cTuning.org/ae